

On the Information Bottleneck Theory of Deep Learning

A reproduction and critical study

Information Theory and Inference — course project

after Saxe, Bansal, Dapello, Advani, Kolchinsky, Tracey & Cox, *J. Stat. Mech.* (2019) 124020

August 7, 2026

Information is surprise

An event of probability p carries

$$\log_2 \frac{1}{p} \quad \text{bits of surprise.}$$

Certain event: 0 bits. Coin flip: 1 bit. One particular pattern out of 4096: 12 bits.

Entropy is average surprise:

$$H(X) = - \sum_x p(x) \log_2 p(x)$$

Equivalently: the average number of yes/no questions needed to pin X down — the length of the best possible code.

Everything in this talk, in these units

fair coin	1.000 bit
our labels ($p = 0.517$)	0.999 bits
12 uniform bits	12.000 bits
a constant	0 bits

A bit is a unit of *uncertainty*. Everything that follows is bookkeeping on uncertainty — and the whole argument will turn on *whose* uncertainty we are counting.

Mutual information: what one variable tells you about another

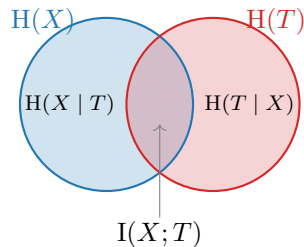
$$I(X;T) = \underbrace{H(X)}_{\text{before}} - \underbrace{H(X|T)}_{\text{after}}$$

The uncertainty about X that knowing T removes.

Equivalently $H(T) - H(T|X)$ or
 $H(X) + H(T) - H(X,T)$ — all three are used later.

Three properties that will matter

- 1 symmetric, ≥ 0 , and $= 0$ exactly when independent;
- 2 **invariant** under any invertible transform of X or T — *the reason to reach for it*;
- 3 **DPI**: if $X \rightarrow A \rightarrow B$ then $I(X;A) \geq I(X;B)$ — processing cannot create information.



The overlap. Property 2 says it does not care what *units* you measure X and T in — keep that in mind for slide 27.

Computing it, case 1: discrete variables — easy

Count the joint table, normalise, sum:

$$\hat{I}(X; T) = \sum_{x,t} \hat{p}(x,t) \log_2 \frac{\hat{p}(x,t)}{\hat{p}(x) \hat{p}(t)}$$

$p(x, t)$	$t=0$	$t=1$	
$x=0$	0.4	0.1	0.5
$x=1$	0.1	0.4	0.5
	0.5	0.5	

$$\Rightarrow I = 0.278 \text{ bits}$$

Marginals are uniform (1 bit each), but the pair agrees 80% of the time — so T answers a bit over a quarter of the question “what is X ?”.

The catch: plug-in bias

With P samples and many distinct values, $\hat{I}$ **overestimates**.

In the limit where every sample is unique, T identifies the sample, hence its label, and

$$\hat{I}(T; Y) = H(Y)$$

no matter what the truth is. Resolution and sample size are not free parameters.

We hit exactly this later: 8 random units at 30 bins “know” 1.0 bit about a label they are independent of.

Computing it, case 2: continuous variables — treacherous

Replace the sum by an integral: $h(X) = - \int p(x) \log_2 p(x) dx$ (*differential entropy*).

Three things change

- 1 it can be **negative** (a narrow Gaussian);
- 2 it is **not invariant**: $h(cX) = h(X) + \log_2 |c|$ — it depends on the units you chose;
- 3 it is not a limit of the discrete entropy — that diverges.

I survives all three: the two $\log_2 c$ terms cancel, so mutual information *is* still invariant. That is why people trust it.

But we never have p — only samples

Three ways out, and every one is an **added assumption**:

- **discretise** $\Rightarrow$ pick a resolution
- **smooth** (KDE) $\Rightarrow$ pick a bandwidth / noise level
- **nearest neighbours** $\Rightarrow$ assume a density exists at all

Hold that thought.
It is the whole talk.

The information bottleneck (Tishby, Pereira & Bialek 1999)

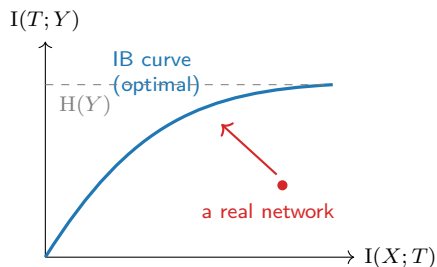
Find a representation T of the input X that is

- as **informative** about the label Y as possible,
- as **compressed** about X as possible.

$$\min_{p(t|x)} I(X;T) - \beta I(T;Y)$$

- $\beta \rightarrow 0$: keep nothing
- $\beta \rightarrow \infty$: keep everything ($T = X$)
- sweeping β traces the **IB curve** — the best $I(T;Y)$ achievable at each $I(X;T)$

It is rate–distortion theory with $I(T;Y)$ as the fidelity measure: a *soft* version of a minimal sufficient statistic.



Everything below the curve is achievable; the curve itself is the frontier. “Learning well” is supposed to mean *moving towards it*.

From a principle to a theory of deep learning (Tishby & Zaslavsky 2015)

A feed-forward network is *already* a Markov chain:

$$Y \rightarrow X \rightarrow T_1 \rightarrow T_2 \rightarrow \cdots \rightarrow T_L \rightarrow \hat{Y}$$

so the DPI applies to it directly, and gives for free

$$I(X; T_1) \geq I(X; T_2) \geq \cdots \geq I(X; T_L), \quad I(T_1; Y) \geq \cdots \geq I(T_L; Y).$$

The proposal

Each layer is a successively compressed representation. Training *should* drive the layers towards the IB curve — and the depth of the network is what lets it get there in feasible steps.

Why anyone should care

- a single, **architecture-agnostic** currency: bits
- an *optimality* target, not just a loss curve
- $I(X; T)$ as a complexity measure $\Rightarrow$ a route to a generalisation bound

An attractive programme. The question is whether the quantity plotted is the one in the theory.

The information plane

Treat each hidden layer T as a summary statistic of the input:

$$Y \rightarrow X \rightarrow T_1 \rightarrow T_2 \rightarrow \dots \rightarrow \hat{Y}$$

Plot, for each layer and each epoch,

$$I(T; Y) \text{ against } I(X; T).$$

Why it is appealing

- architecture-agnostic — a common currency
- the IB principle gives an optimal frontier

$$\mathcal{L} = I(X; T) - \beta I(T; Y)$$

Reading the plot

- right = *fitting* (learns about input)
- ↑ up = *fitting* (learns about label)
- ← left = **compression**

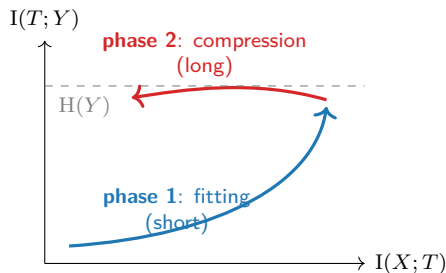
Shwartz-Ziv & Tishby 2017: what they saw

The experiment

- 12 binary inputs, 4096 patterns, binary label
- network 12-10-7-5-4-3-2, tanh
- SGD; 50 repetitions averaged
- MI by **binning** each unit into 30 bins on $[-1, 1]$

The observation

- every layer sweeps *right and up* — fitting — for a few hundred epochs
- then turns and moves *left* for thousands of epochs — **compression**
- deeper layers compress more
- the turn coincides with the gradient SNR collapsing: “drift” $\rightarrow$ “diffusion”



One layer's trajectory, schematically. *This picture is the entire object of the dispute* — and we will reproduce it exactly.

Three claims (Shwartz-Ziv & Tishby 2017)

Claim 1 — Two phases

Training has a *fitting* phase, then a distinct **compression** phase.

Claim 2 — Causality

The compression phase **explains** why deep networks generalise.

Claim 3 — Mechanism

Compression is driven by the **diffusion** of stochastic gradient descent.

Saxe et al.: **none of the three holds in general.**
This project reproduces that argument from scratch.

The problem: deterministic networks carry infinite information

For a deterministic continuous layer $h = f(X)$:

$$I(h; X) = H(h) - \underbrace{H(h | X)}_{\text{point mass} \Rightarrow -\infty}$$

$$I(h; X) = \infty \quad \text{at every point in training}$$

A constant has no fitting phase and no compression phase.

So every finite number ever plotted comes from an **added assumption**:

- **binning** $T = \text{bin}(h) \Rightarrow H(T | X) = 0, I(T; X) = H(T)$
- **noise** $T = h + Z, Z \perp X \Rightarrow I(T; X) = H(T) - H(Z)$

Neither is part of the network. Both are choices made by the analyst.

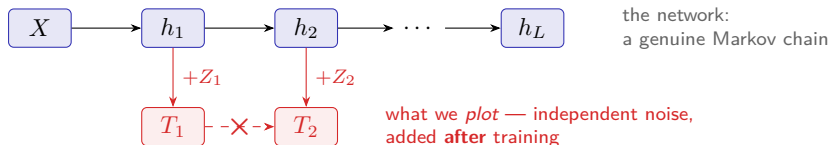
The two tricks — and the price of each

	Binning	Noise
definition	$T = \text{bin}(h)$	$T = h + Z, \quad Z \perp X$
what it buys	$H(T \mid X) = 0$, so $I(T; X) = H(T)$	$I(T; X) = H(T) - H(Z)$
what you must choose	a resolution (bin edges + count)	a noise level σ_{MI}^2
ceiling	$I \leq \log_2 P$ — set by the estimator, not the network	set by σ_{MI}^2 relative to $\ h\ $
the price	move the edges and the conclusion moves	rescale h and the answer moves, though the function is unchanged

Both replace “how much does the layer know about X ?” with
*“how much does the layer know about X , **at the resolution I chose?**”*

A third route — k -nearest-neighbour — avoids fixing σ_{MI}^2 by using only $\Delta I = \Delta H$, at the price of a different assumption. We will watch that one fail too.

Two immediate casualties



The DPI does not apply

T_2 is **not** a function of T_1 , so no inequality relates $I(X; T_1)$ to $I(X; T_2)$.

The layer ordering of the plane is not guaranteed.

our ReLU run, final $I(X; T)$ [bits]:

8.30 $\rightarrow$ 4.58 $\rightarrow$ 2.29 $\rightarrow$ 3.12 $\rightarrow$ 4.15

it *rises with depth* — impossible for $I(X; h_l)$.

Invariance is destroyed

$I(X; g(h)) = I(X; h)$ for invertible g — the very reason to reach for I.

$T = h + Z$ is **not** invariant: Z enters at a *fixed* scale.

rescale the weights, keep the function:

4.32 bits $\rightarrow$ 0.03 bits

same map throughout (slide 27).

Root of both: $I(X; T)$ compares the representation's *scale* to a resolution **we** chose.

Training moves the first; the second is a constant we picked.

Four estimators, all implemented and validated

estimator	assumption	key formula
Binning	discretise h	$I(T; X) = H(T) = -\sum_i p_i \log p_i$
KDE bounds	$T = h + \mathcal{N}(0, \sigma^2)$	$I \leq -\frac{1}{P} \sum_i \log \frac{1}{P} \sum_j e^{-\ h_i - h_j\ ^2 / 2\sigma^2}$
Kraskov k -NN	$T = h + Z$, any Z	$I = H(T) - c$; use k -NN distances
Exact (linear)	$T = \bar{W}X + \epsilon$	$I = \frac{1}{2} \log \bar{W}\Sigma_X\bar{W}^\top + \sigma_{MI}^2 I - \frac{1}{2} \log \sigma_{MI}^2 I $

Validation — 46 checks, all passing

- agreement with the **original authors' code** to relative 10^{-10}
- analytic cases exact: $\log_2 P$ codes, Gaussian channels, $H(U[0, 1]^d) = 0$
- cross-estimator: exact value lies inside the KDE bounds
- NumPy vs PyTorch trainers agree to 10^{-7} ($17\times$ faster)

The dataset makes the measurement exact

12 binary inputs $\Rightarrow$ all $2^{12} = 4096$ patterns, **each exactly once**.

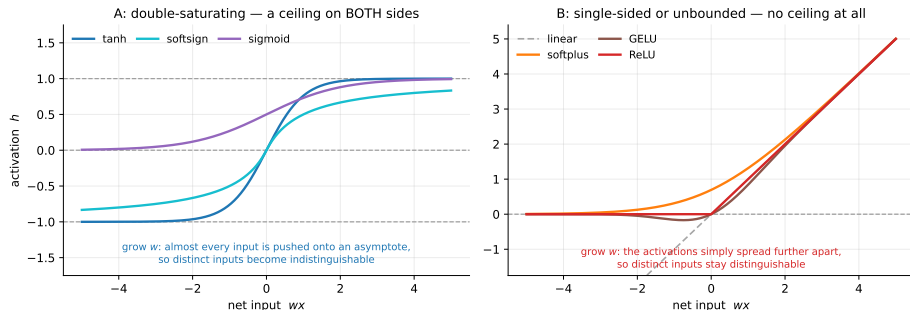
- the input alphabet is finite and *fully enumerated*
- taken uniform: $H(X) = \log_2 4096 = 12$ bits *exactly*
- binning MI over all 4096 patterns has **no sampling error**

Network 12-10-7-5-4-3-2,
final layer two sigmoidal units.

So any effect we see is not noise

$H(Y) = 0.9992$ bits — the ceiling for $I(T; Y)$.

The distinction that turns out to matter



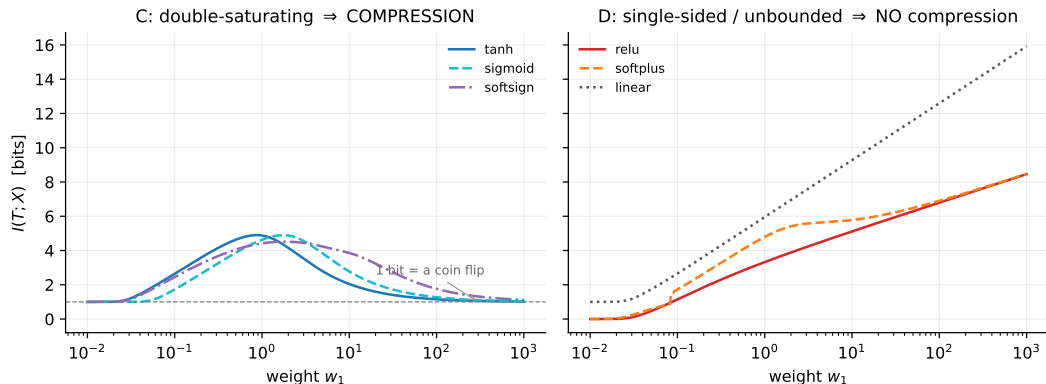
Not “linear vs nonlinear” — whether there is a **ceiling on both sides**. A network starts small and *must grow its weights* to compute anything nonlinear; panel A shows what that does to a bounded unit, panel B that it does nothing of the kind to an unbounded one.

None of this is a property of the learning algorithm.

	bdd. below	bdd. above	compression
tanh	yes	yes	0.69 bits
softsign	yes	yes	0.11 bits
ReLU	yes	no	0.10 bits
softplus	yes	no	0.01 bits

KDE, $\sigma^2 = 0.1$, 5 seeds. The ordering tracks *how fast* a unit saturates, not merely whether it is bounded.

An exactly solvable model explains everything

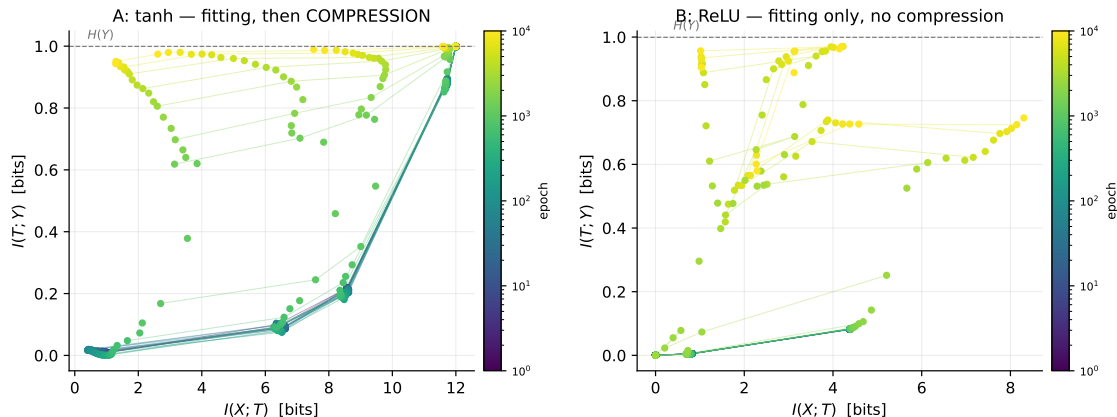


$X \sim \mathcal{N}(0, 1)$, $h = f(w_1 X)$, $T = \text{bin}(h)$ — computed *exactly* from the Gaussian CDF.

- **tanh**: rises, peaks, falls to *exactly* 1 bit (a coin flip: only the sign survives)
- **ReLU**: rises without bound; slope is *half* the linear one (half the inputs sit in the zero bin)

Networks start small and must grow their weights $\Rightarrow$ tanh traverses this curve during training.

The real network: change one word, lose the phenomenon



Identical data, identical training, identical measurement. Only $\tanh \rightarrow \text{relu}$.

Test accuracy: 96.2 % vs 97.3 % — the ReLU network is *better*.

The one ReLU curve that compresses is the **sigmoidal output layer**.

Two of the three claims, already gone

Claim 1 — two phases — NO

One identifier changed, $\tanh \rightarrow \text{relu}$, and the compression phase is gone: 10.66 bits $\rightarrow$ 0.47 bits.

It is not a universal property of learning. It is a **side effect of the nonlinearity** — exactly what the one-neuron model predicted two slides ago.

Claim 2 — compression causes generalisation — NO

	compression	test acc.
$\tanh$	10.66 bits	96.2 %
ReLU	0.47 bits	97.3 %

The network that compressed *nothing* generalised **better**.

The exception that proves the rule

One curve in the ReLU plane *does* hook left — the **output layer**. And the output layer is not ReLU: it is **sigmoidal**, because it must produce probabilities in $[0, 1]$. Double-saturating, exactly like $\tanh$.

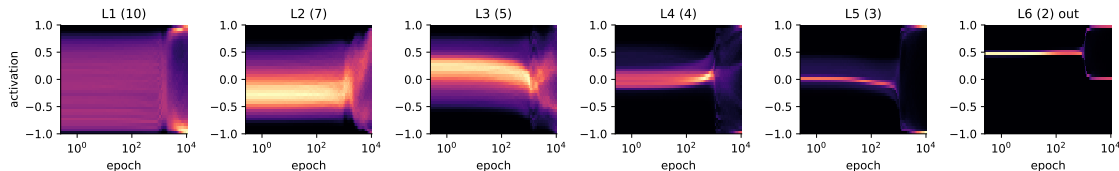
ReLU net	L1	L2	L3	L4	L5	out
compression	0.00	0.00	0.00	0.02	0.08	0.37

The five ReLU layers compress 0.10 bits *between them*; the one sigmoid layer compresses 0.37 — 79 % of the network's total.

The mechanism predicts its own exception, and the exception is there. Claim 3 — that SGD's noise is the *cause* — is still standing; we remove that next.

The mechanism, caught in the act

E1: tanh activations migrate into saturation



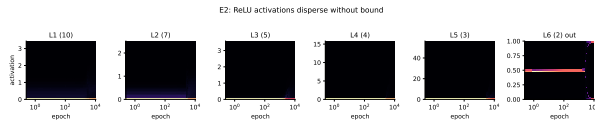
tanh: fraction of layer-5 units saturated
($|h| > 0.98$)
 $0.0\% \rightarrow 75.4\%$

ReLU: max activation
grows without bound to 56.3

How to read those heatmaps — and the ReLU contrast

One panel per layer, L1 to the output.

- **x**: epoch, log scale
- **y**: the raw activation h , penned into $[-1, 1]$ by $\tanh$
- **colour**: where the units actually are — bright = most of them, dark = empty



the same plot for ReLU — note the vertical axis

What the two phases look like

Early: one bright band at $h \approx 0$. Weights are still small, so the data passes through the *linear* middle of $\tanh$ undistorted.

Late: the band **splits and migrates** to $+1$ and -1 . The weights have grown, and the units are saturating.

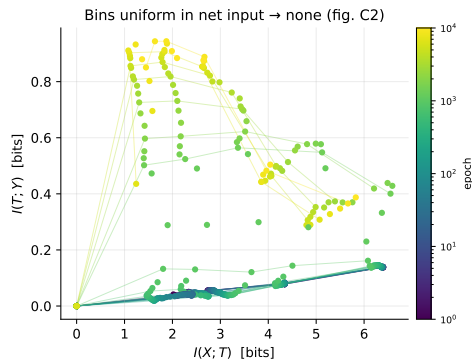
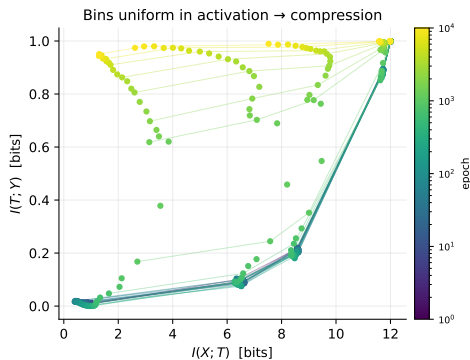
Why one gives a compression phase and the other does not

Three quarters of layer 5 now sits in the outermost two bins, so the binner assigns all of it the *same two symbols*. $H(T)$ collapses. That mechanical pile-up against a fixed wall is what gets read as “a massive loss of information”.

ReLU has **no wall**. As its weights grow the activations simply expand outward — up to 56.3 here — spreading across *more* bins, so the estimate never falls.

Layer 5, fraction with $|h| > 0.98$: 0.0% $\rightarrow$ 75.4%.

It is the measurement, not the network

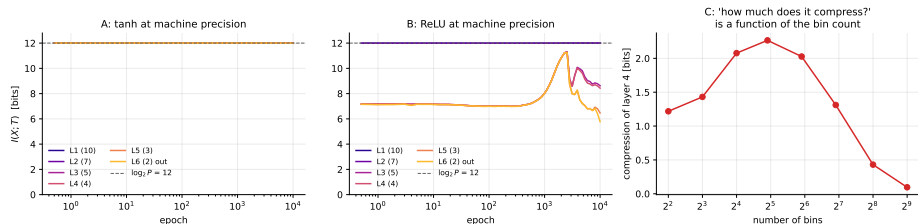


Same network. Same weights. Same training run.
Only the bin edges differ.

layer 5 compression: 5.59 bits $\rightarrow$ 0.32 bits

At full machine precision every input gets its own code: $I(X; T)$ pinned at 12 bits, no dynamics at all.

Take the bins away entirely: there are no dynamics at all



Stop rounding: two activations share a symbol only if their **float32 bit patterns are identical** — $\sim 2^{32}$ levels, which is what the network actually computes in.

The layer is then an **injective** map — a perfect ID tag for the input — so $I(X; T) = H(X) = 12$ bits at every epoch, in every layer (panel A). **No fitting phase, no compression phase, no dynamics: a flat line.**

ReLU (panel B) does not sit at 12: a unit that is off outputs **exactly** zero, so some patterns genuinely collide — a real atom in the distribution, and the reason the k -NN estimator breaks later.

And this is not a *better estimate* of the same thing — for a deterministic network it is **the true value**. Slide 11 said so in advance: constant for discrete X , infinite for continuous X . Here is the constant, measured.

Panel C sweeps everything in between: one layer's measured compression runs from 0.10 to 2.27 bits, *not even monotone* in the bin count.

Removing the estimator entirely

So far every number has depended on a choice. Take the choice away.

The model (paper §3) — a linear student and a linear teacher:

$$X \sim \mathcal{N}(0, \tfrac{1}{N_i} I), \quad Y = W_o X + \epsilon_o$$

$$T = \bar{W} X, \quad \bar{W} = W_l \cdots W_1$$

Deterministic again, so $I(T; X) = \infty$. *For the analysis only*, set $T = \bar{W} X + \epsilon_{MI}$, $\epsilon_{MI} \sim \mathcal{N}(0, \sigma_{MI}^2 I)$.

Everything is now jointly Gaussian, hence **closed form**:

$$I(T; X) = \tfrac{1}{2} \log |\bar{W} \Sigma_X \bar{W}^\top + \sigma_{MI}^2 I| - \tfrac{1}{2} \log |\sigma_{MI}^2 I|$$

$$I(T; Y) = H(T) + H(Y) - H(T, Y)$$

and so is the generalisation error E_g itself.
paper eqs. (6), (G.1)–(G.4) and (5)

Is a *linear* network a fair test?

Yes — it is linear in the *input*, not in the learning:

- the optimisation is still **non-convex**, the dynamics still nonlinear in time (Saxe, McClelland & Ganguli 2014)
- it genuinely **overtrains** (Advani & Saxe 2017)
- a linear map can correlate neurons and project directions out — the two mechanisms usually offered for compression

What is gone

No bins, no bandwidth, no neighbours. E_g is the *exact expected* error, not a test-set estimate. One assumption remains, σ_{MI}^2 , and it is written down.

We look for one thing: **any leftward motion at all**.

No compression — at any depth, under any optimiser

Top row (paper fig. 3) — full-batch GD, a network that *generalises*.

Bottom row (paper fig. 4) — SGD, $b = 5$, at $P = N_i$: the interpolation peak, where linear networks overtrain worst.

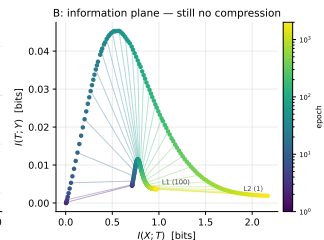
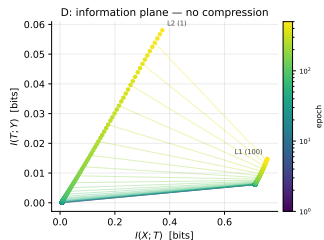
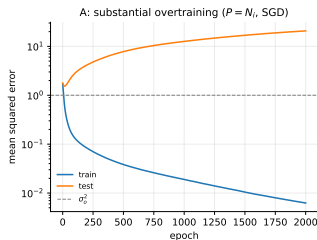
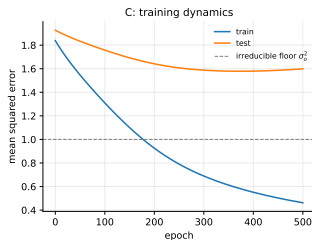
Left panels are the learning curves; right panels are the **exact** information plane.

	compression	final E_g
generalises	0.000 00 bits	1.61
overtrains	0.000 00 bits	20.30

irreducible floor $\sigma_o^2 = 1.00$

Both planes run monotonically **right**. Zero leftward motion — any layer, up to six hidden layers, SGD or BGD, 500 or 20 000 epochs.

Panel **B** *does* show the overtraining — on the **y**-axis: $I(T; Y)$ rises, then falls as the network memorises. The **x**-axis never turns: the plane sees the failure, but the *compression* coordinate is not what registers it.



The dissociation

network	compression	compresses?	gap	generalises?
tanh, 80 % data	10.66 bits	yes	0.019	yes
ReLU, 80 % data	0.47 bits	no	0.014	yes
tanh, 30 % data	7.99 bits	yes	0.063	no
ReLU, 30 % data	0.62 bits	no	0.055	no
linear, BGD	0.00 bits	no	1.6×	yes
linear, SGD $b=5$	0.00 bits	no	20.3×	no

All four combinations occur.

Neither implies the other.

Same function, different “information”

Take $\hat{y} = w_2 w_1 x$ and rescale

$$\tilde{w}_1 = w_1/c, \quad \tilde{w}_2 = c w_2.$$

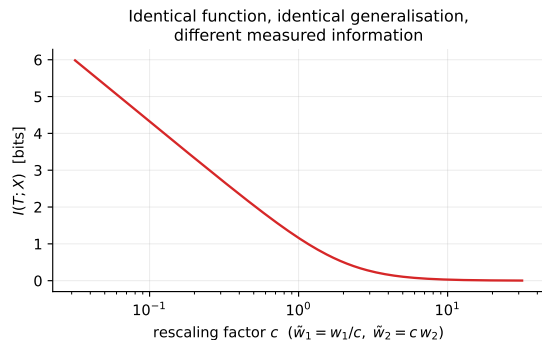
Every member computes the **same map** $\Rightarrow$ generalises identically.

But

$$I(T; X) = \frac{1}{2} \log \frac{w_1^2/c^2 + \sigma_{MI}^2}{\sigma_{MI}^2}$$

depends on c .

4.32 bits $\longrightarrow$ 0.03 bits
for an *unchanged* input–output map



Remove the randomness entirely

Full-batch gradient descent: no sampling noise, no diffusion, deterministic updates.

	L1	L2	L3	L4	L5	L6	total
tanh, SGD	0.00	0.02	0.23	2.27	5.59	2.56	10.66
tanh, BGD	0.01	1.34	3.30	3.86	6.52	2.84	17.87
ReLU, SGD	0.00	0.00	0.00	0.02	0.08	0.37	0.47
ReLU, BGD	0.00	0.00	0.00	0.03	0.11	0.00	0.13

Deterministic training compresses *more*.

The prediction fails — in the wrong direction.

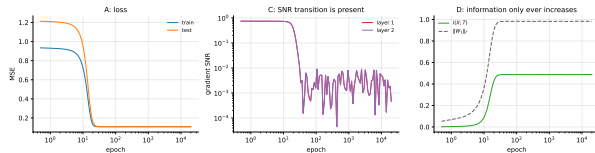
The SNR transition is real — and irrelevant

$$\text{SNR}_l = \frac{\|\langle \partial E / \partial W_l \rangle\|_F}{\|\text{STD}(\partial E / \partial W_l)\|_F}$$

Both networks lose 20 to 68× of SNR. No systematic difference between the compressing and non-compressing one.

Why it is generic

At a minimum the *mean* gradient vanishes **by definition**, while per-sample disagreement does not.



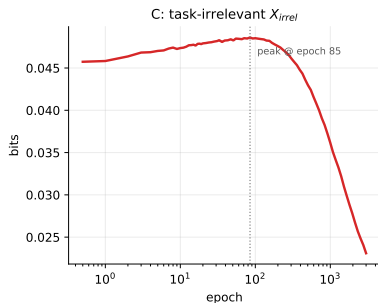
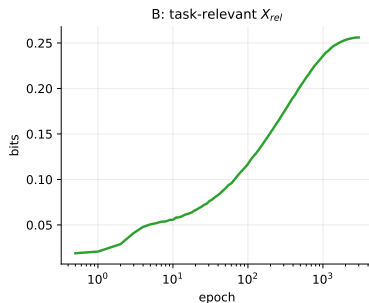
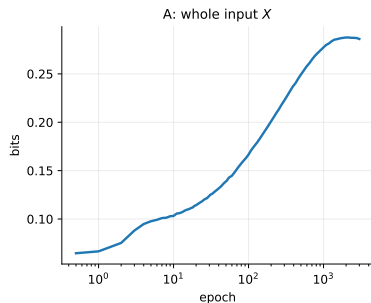
A 1–1–1 linear net: compression **impossible by construction**, yet SNR falls 16 502×.

Also: the theoretical step from “max-entropy *across training runs*” to “maximises $H(X | T)$ *for one run*” conflates two different random variables.

The steelman: make part of the input genuinely useless

$X = [X_{\text{rel}} \ X_{\text{irrel}}]$, teacher weights on X_{irrel} set to **exactly zero**.

A network that generalises *must* learn to ignore it.

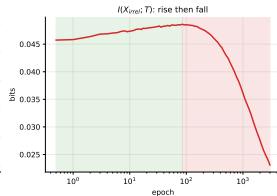
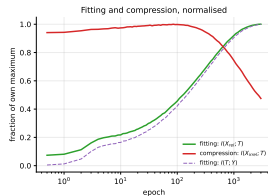


A whole input: **no** compression **B** relevant: rises **C** irrelevant: **compresses 53 %**

But *when*? — the claim was a second phase

Measured timing

- compression **begins**: epoch **85**
- at that moment, fitting is only **39 %** complete
- fitting reaches 90 % at epoch 995 — 10× later



Interleaved, not sequential.

Why is it invisible in $I(X; T)$? Sub-additivity: $I(X; T) = I(X_{\text{rel}}; T) + I(X_{\text{irrel}}; T) + \underbrace{I(X_{\text{rel}}; X_{\text{irrel}} | T)}_{\geq 0}$

Three things we add

1. The timing, quantified

The paper asserts fitting and compression are concurrent. We measure it: compression starts at 39 % of fitting (33 %, 7 % for other seeds).

2. The Kraskov estimator is *invalid* for ReLU — next slide

Its validity assumption (an absolutely continuous distribution) is quietly false for ReLU layers, and the estimate it returns is a function of the guard constant ε rather than of the data. *The paper's appendix B.3 does not flag this.*

3. Two inconsistencies in the paper's own specification

Σ_X omitted from eqs (5), (6), (G.1)–(G.3) and a missing $\frac{1}{2}$ in (6);
figures 3 and 4 described with identical parameters but opposite conclusions.

The estimator can invent a phase that is not there

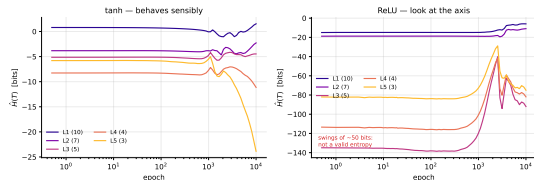
The k -NN route needs no noise level: with $T = h + Z$, $I(T; X) = H(T) - H(Z)$, so *changes* in H are changes in I . Estimate H from neighbour distances (eq. B.10):

$$\hat{H} = \frac{d}{P} \sum_i \log(r_i + \varepsilon) + \dots$$

It assumes an **absolutely continuous** distribution.

A ReLU unit that is off outputs *exactly* zero, and units switch off together — so activation vectors are **bit-identical**, the distribution has an **atom**, and differential entropy is not even defined. Then

$r_i = 0$, the guard $\varepsilon = 10^{-16}$ takes over, and each duplicated point contributes $\frac{d}{P} \log \varepsilon$.



layer	dupl.	$\hat{H}$	ε alone
tanh L3–L5	0 %	−4.5 / −11.1 / −23.9	0
ReLU L3	32.5 %	−92.0	−86.4
ReLU L4	35.3 %	−81.9	−75.0
ReLU L5	50.2 %	−75.0	−80.0

The guard term **alone** reproduces the ReLU values: the estimator is reporting ε , not the data. The duplicate count moves during training, so the “entropy” moves with it — a 55 bit to 98 bit **compression phase in the network that does not compress**.

Extension: what about transformers?

The paper's mechanism is about the *activation function* — and modern architectures use non-saturating ones. But a transformer block has two **bounded** operations the paper never examined:

- **softmax attention** — outputs live in the convex hull of the value vectors; attention weights saturate towards one-hot
- **LayerNorm** — projects the residual stream onto a sphere of fixed radius $\sqrt{d}$, so it *cannot* spread out

Prediction from the paper's own mechanism

A transformer should compress because of its **normalisation and attention**, not its feed-forward nonlinearity — so the LayerNorm axis should matter more than the activation axis. Removing LayerNorm should remove the compression.

Tested on the *same* 4096 twelve-bit patterns, fed as 12 tokens, so binning MI stays exact and comparable.

Result: the normalisation axis dominates

feed-forward	with LayerNorm	without	LN effect
GELU	2.47 bits	0.01 bits	+2.45
ReLU	3.77 bits	0.07 bits	+3.70
tanh	3.68 bits	2.15 bits	+1.53
mean effect of LayerNorm			+ 2.56
spread across activations			1.31–2.14

- **GELU:** 0.01 bits **without LayerNorm**, 2.47 bits **with it**. A purely non-saturating architecture compresses only once you normalise it.
- tanh is the only activation still compressing without LN — the paper's mechanism showing through.

Two surprises, and a sharper statement of the mechanism

It shows up in the FFN, not attention

LayerNorm *causes* it but is not *where* it appears: it sits on the FFN's input, capping the scale the FFN sees.

Removing LN: attention saturates *harder*, compresses *less*

attention entropy $3.56 \rightarrow 0.43$ bits (nearly one-hot)

... but residual norm explodes $2.1 \rightarrow \approx 100$.

What causes apparent compression is not “saturation” loosely construed — it is a representation whose **scale stops growing while the weights keep growing**.

tanh has this via its asymptotes; LayerNorm by construction; **softmax attention does not**, because the vectors it averages can still grow without bound.

- ① **Two phases:** **no** in general. Compression tracks *double-saturating* nonlinearities. One identifier changed removes it — and improves accuracy.
- ② **Causality:** **no**. All four (compresses, generalises) combinations occur; identical information planes with $20\times$ different generalisation.
- ③ **SGD diffusion:** **no**. Deterministic BGD compresses *more*; the SNR transition appears where compression is impossible.
- ④ **The deeper issue is definitional.** $I(h; X) = \infty$ for a deterministic network. Every finite number is a joint property of the network *and* an arbitrary assumption.

But compression is not an empty idea. When part of the input really is irrelevant, the network *does* discard it — concurrently with fitting, and invisibly to $I(X; T)$. The framing survives; the aggregate statistic and the two-phase story do not.

What was built

Library `ibdl/`

- 4 MI estimators from the paper's equations
- exactly solvable minimal model
- instrumented trainers (PyTorch + fast NumPy)
- tiny transformer
- 46 validation checks, all passing

Notebooks

Presentation cut — self-contained, no library:

- 1 the whole argument, end to end
- 2 the transformer extension

Research record — one per paper section:

- 01 minimal model (figs. 2, C1)
- 02 replication (figs. 1, A1, B1, B5, C2, C3, E1, E2)
- 03 linear networks (figs. 3, 4AB, F1, H1, C.5)
- 04 SGD vs BGD (figs. 5, 4CD, I1, I2)
- 05 simultaneous fitting (fig. 6)
- 06 MNIST + KDE (figs. 1CD, B2, B3)
- 07 transformer extension

Thank you

Questions?